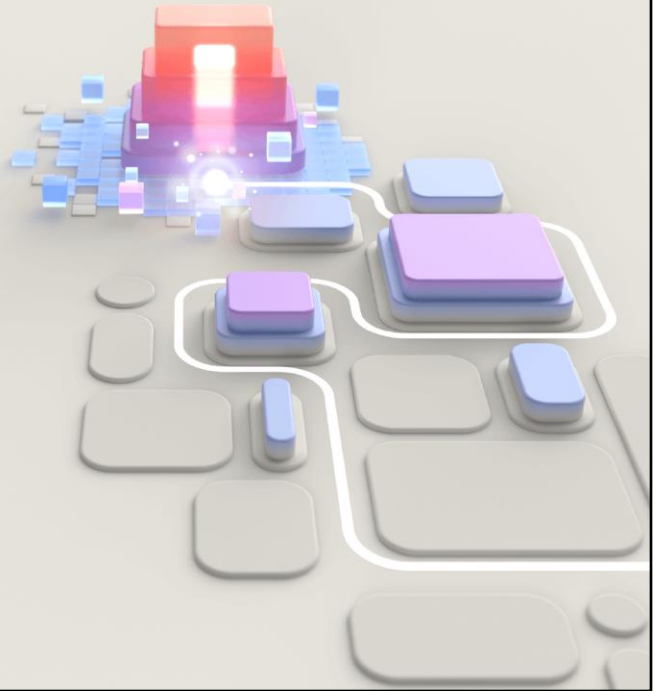




PL-400T00 Microsoft Power Platform Developer

© Copyright Microsoft Corporation. All rights reserved.



Custom Connectors

© Copyright Microsoft Corporation. All rights reserved.



Modules



- Get started with custom connectors
- Azure API Management and custom connectors
- OpenAPI definitions
- Policy templates
- Write code in a custom connector

© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Get started with custom connectors
2. Azure API Management and custom connectors
3. Open API definitions
4. Policy templates

Module 1: Get started with custom connectors

© Copyright Microsoft Corporation. All rights reserved.

In this module, you'll:

- Learn about the role of custom connectors.
- Build a connector

Learning Objectives

After completing this module, you will be able to:

- 1 Get started with custom connectors in Power Automate
- 2 Configure custom connectors with authenticated APIs in Power Automate

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Get started with custom connectors in Power Automate

<https://learn.microsoft.com/training/modules/get-started-custom-connector>

Configure custom connectors with authenticated APIs in Power Automate

<https://learn.microsoft.com/training/modules/configure-custom-connectors-api>



© Copyright Microsoft Corporation. All rights reserved.

Get started with custom connectors in Power Automate

[https://learn.microsoft.com/training/modules/get-](https://learn.microsoft.com/training/modules/get-started-custom-connector)

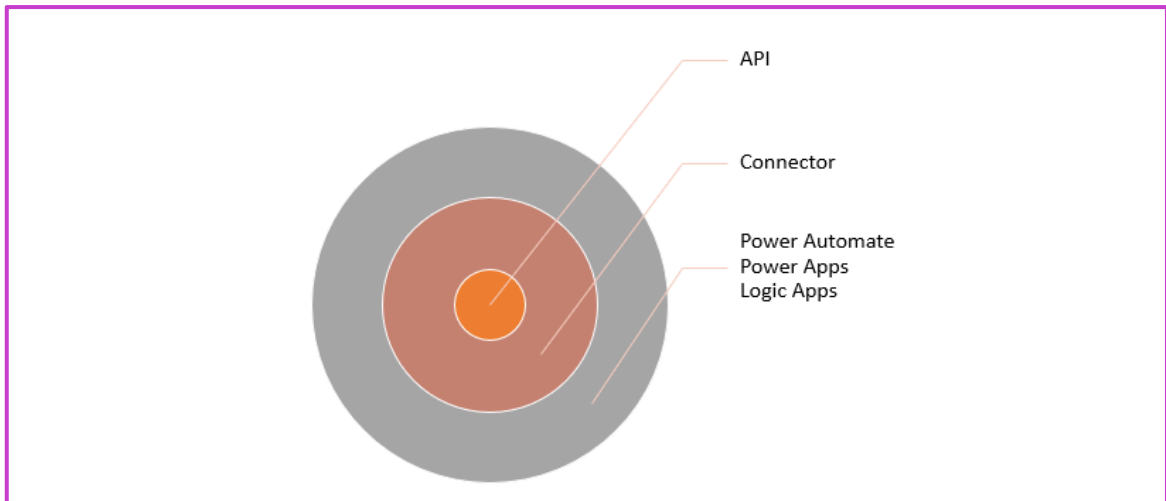
[started-custom-connector](https://learn.microsoft.com/training/modules/get-started-custom-connector)

Configure custom connectors with authenticated APIs in Power Automate

[https://learn.microsoft.com/training/modules/configure](https://learn.microsoft.com/training/modules/configure-custom-connectors-api)

[-custom-connectors-api](https://learn.microsoft.com/training/modules/configure-custom-connectors-api)

Introduction to custom connectors



© Copyright Microsoft Corporation. All rights reserved.

Connectors help make it easier for app and flow makers to connect to other apps, data, and devices in the cloud in the following ways:

- Connection is done in a consistent, repeatable way that is discoverable by makers.
- Connectors have actions that allow makers to control when an operation is performed.
- Connectors can have triggers that allow automation to start when the triggering event occurs.

Custom connector lifecycle

A custom connector is a wrapper around a REST API that lets an underlying service communicate with Power Apps, Power Automate, or Azure Logic Apps.



© Copyright Microsoft Corporation. All rights reserved.

A connector is a proxy, or a wrapper, around an API that allows Power Automate, Power Apps, and Logic Apps to communicate with the underlying service. Many connectors are predefined and available for use without you having to define anything. These built-in connector definitions include services from Microsoft and others. When the service or API that you want to use doesn't have a prebuilt connector, you can create your own custom connector. If a prebuilt connector doesn't expose all triggers and actions that you need from the API, you can build a custom connector in addition to the available one to handle these extra requirements. After the custom connector has been defined, it becomes available for use when you're building apps and flows in that environment, similar to the built-in connectors.

Identify the API

You can use any language and platform for your API, as long as it's made available as a REST API

© Copyright Microsoft Corporation. All rights reserved.

Your first task is to check the connector reference to determine if a suitable connector already exists for the API/service. If a suitable connector doesn't already exist, then you can make your own custom connector. You can define the custom connector to describe any REST API that is public (one that is available on the internet) or private (one that is only on your internal network). Private APIs require the on-premises data gateway to access internal resources.

Build the API

Azure Functions can be used to create APIs that can be described by a custom connector

© Copyright Microsoft Corporation. All rights reserved.

If a compatible API doesn't already exist, developers can build one by using any technology of choice that is capable of hosting a REST API, including Microsoft Azure Functions and Azure App Services. You can also use Azure API Management to increase manageability and governance of the APIs.

Describe the API

You can create custom connectors using different approaches:

- Custom Connector wizard
- Import OpenAPI definition
- Import Postman collection
- Import from GitHub
- Create from Azure API Management

© Copyright Microsoft Corporation. All rights reserved.

For Power Apps and Power Automate to discover what triggers and actions are available from the API, you must identify them when you create your custom connector. You can accomplish this task by manually editing the connector definition in the maker portal or by importing a definition in a supported format. This process includes identifying the authentication requirements for the API.

To define a connector's actions and triggers, you need to describe the connector. A custom connector can be defined by describing only the API triggers and actions that you need. For example, if an API has 100 actions and 10 triggers that are available, you could build a custom connector with a single action. You can also describe later as your needs change.

When you describe the API, you'll use at least one of the following approaches. If you choose to import either an OpenAPI definition or a Postman collection, you can still manually manipulate your connector's definition in the portal.

- **Manually create in the portal** - When using this approach, you would use the API documentation and examples of inputs and outputs and then define each action and trigger. This approach is good for simple APIs where you don't have the ability to import the definition. You can also use this approach to refine the definitions after you import them.
- **Import OpenAPI definition** - [OpenAPI](#) is a specification that describes REST, simplifying the tasks of developing and consuming an API by effectively mapping all resources and operations that are associated with it. Many services/APIs publish an OpenAPI definition, and you can import it to describe the API in the custom connector. Importing version 2.0 is supported, but version 3.0 isn't currently supported. This approach is beneficial if an OpenAPI definition is available from the API publisher or, if you're developing the API, you can publish one.
- **Import Postman collection** - [Postman](#) is a tool for testing REST APIs. You can use this tool to perform the actions on the API that you want to describe and then save them as a [Postman collection](#). This collection is then imported to describe the API in the custom connector. Currently, you can use this approach with v1, V2.0, or V2.1 collections.
- **Import from GitHub** - Import connectors from the [Power Platform Connectors repository](#). This is the best approach when there's already a connector published in the repository and you want to import it to modify or use it as a custom connector.
- **Export from Azure** - If your API is built as an Azure App Service, Microsoft Azure Functions, or is managed by Microsoft Azure API Management, you can export from those services and it will automatically create the initial custom connector definition.

Use the custom connector

You can use your custom connector the same way that you use any built-in connectors

© Copyright Microsoft Corporation. All rights reserved.

You can use your custom connector the same way that you use any built-in connectors. You create the connection to the API and call any operations provided by the API the same way that you call operations for built-in connectors.

Connectors created in Power Apps or Power Automate can be used in both services, but connectors created in Azure Logic Apps cannot be used directly in other services. However, a connector can be easily recreated using the same OpenAPI definition or Postman collection used to create the Azure Logic Apps connector.

For more information about using custom connectors from Power Apps, Power Automate, and Azure Logic Apps:

- [Use a custom connector from a Power Apps app](#)
- [Use a custom connector from a flow](#)
- [Use a custom connector from a logic app](#)

Share the custom connector

Connectors can be shared with other users in your organization the same way you share resources in Power Apps, Power Automate, and Azure Logic Apps.

© Copyright Microsoft Corporation. All rights reserved.

Connectors can be shared with other users in your organization the same way you share resources in Power Apps, Power Automate, and Azure Logic Apps. For more information, see [Share a custom connector in your organization](#).

If you want to share the connector with all users in Power Apps, Power Automate, and Azure Logic Apps, you can submit the connector for Microsoft certification. During this process, Microsoft reviews the connector, checks for technical and content compliance, and validates functionality for Power Apps, Power Automate, and Azure Logic Apps. For more information, see [Submit your connector for Microsoft certification](#).

Demo: Create a custom connector

DEMO

© Copyright Microsoft Corporation. All rights reserved.

Use the Wizard or import an Open API definition

Configure API security

How a custom connector authenticates to the API

- **No authentication**
- **Basic authentication** – Username and password
- **OAuth 2.0** – Generic OAuth 2.0 or prebuilt configurations for Azure AD, GitHub, Slack, etc.
- **API key** – For example an Azure Function

© Copyright Microsoft Corporation. All rights reserved.

When you define the custom connector, you'll identify how it should authenticate to the API.

- **No authentication:** This choice is for APIs that allow anonymous access, and it's the default if you don't configure a specific authentication type.
- **Basic authentication:** This choice is for APIs that require a user and password to be sent with each request.
- **OAuth 2.0:** OAuth 2.0 is the industry-standard protocol for authorization. This option can be configured for any API that uses OAuth 2.0. Prebuilt configurations are available for you to select from for Microsoft services like Azure Active Directory (Azure AD), GitHub, and Azure DevOps. Similar prebuilt configurations are available for other services like Dropbox, Slack, Stripe, and more. The prebuilt configurations simplify what you need to provide to configure the identity provider.
- **API key:** An API key is a token that a client provides when making API calls. The API publisher typically provides the key, and it must be included in each request from the custom connector.

<https://learn.microsoft.com/connectors/custom-connectors>

Create an Azure Function to be used in a custom connector

Create the Azure Function with the HTTP trigger

Authenticate using either

- API Key
- Azure AD

Create the custom connector either by:

- Creating manually with the custom connector wizard
- Adding the Azure Function to Azure API Management

© Copyright Microsoft Corporation. All rights reserved.

Azure Functions can be used for business logic and can easily be consumed as a custom connector

The Azure Function must use the HTTP trigger. After the Azure Function has been created it will generate the URL endpoint and API Key.

You can authenticate either by using the API key or by using Azure Active Directory.

You can then create the custom connector by using APIM which is described in the next module or manually by using the custom connector wizard

<https://learn.microsoft.com/connectors/custom-connectors/create-custom-connector-aad-protected-azure-functions>

Module 2: Azure API Management and custom connectors

© Copyright Microsoft Corporation. All rights reserved.



In this module, you'll:

Learning Objectives

After completing this module, you will be able to:

- 1 Create a custom connector
- 2 Use a custom connector in a Power Apps app to call a web API

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Discover and use Web APIs with Power Apps

<https://learn.microsoft.com/training/modules/discover-web-apis-power-apps>



© Copyright Microsoft Corporation. All rights reserved.

Discover and use Web APIs with Power Apps

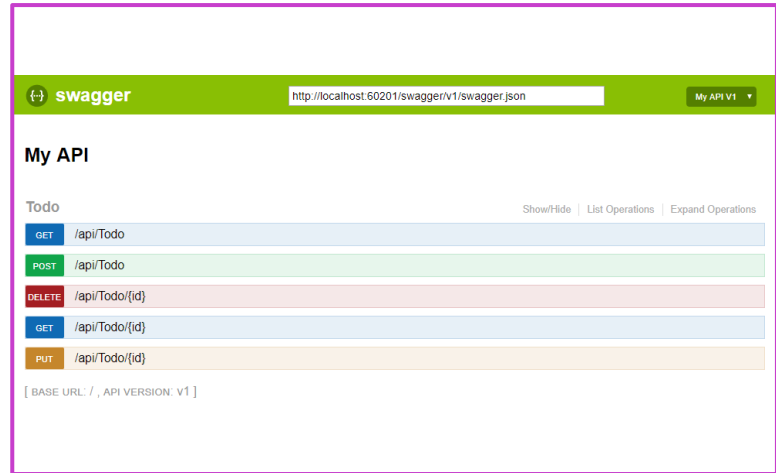
<https://learn.microsoft.com/training/modules/discover->

[web-apis-power-apps](https://learn.microsoft.com/training/modules/discover-web-apis-power-apps)

Azure API Management

Azure API Management helps organizations publish web APIs to external, partner, and internal developers to unlock the potential of their data and services.

But before publishing the web APIs to be consumed by other developers, the author of the web APIs first must deploy them to Azure API Management.



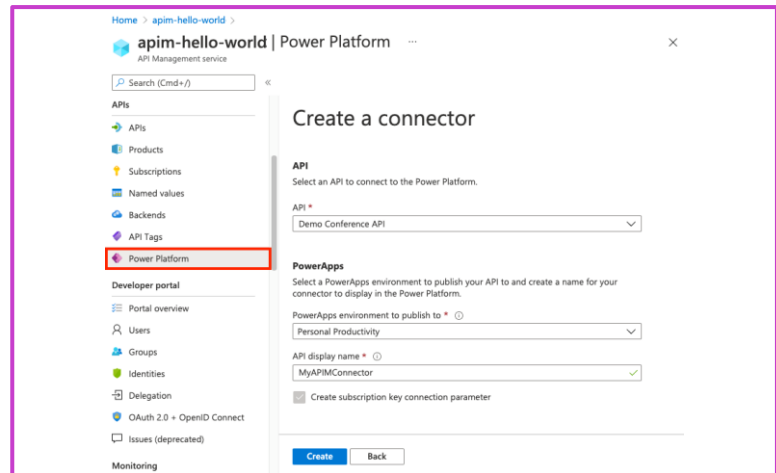
© Copyright Microsoft Corporation. All rights reserved.

You can control access to APIs including Azure Functions with APIM

<https://learn.microsoft.com/training/modules/integrate-openapi-enabled-web-api-with-apim-and-visual-studio/>

Create a custom connector from Azure API Management

- Select environment
- Enter connector name



© Copyright Microsoft Corporation. All rights reserved.

Once you have added APIs to API Management it is simple to create a custom connector from the Azure portal

<https://learn.microsoft.com/azure/api-management/export-api-power-platform>

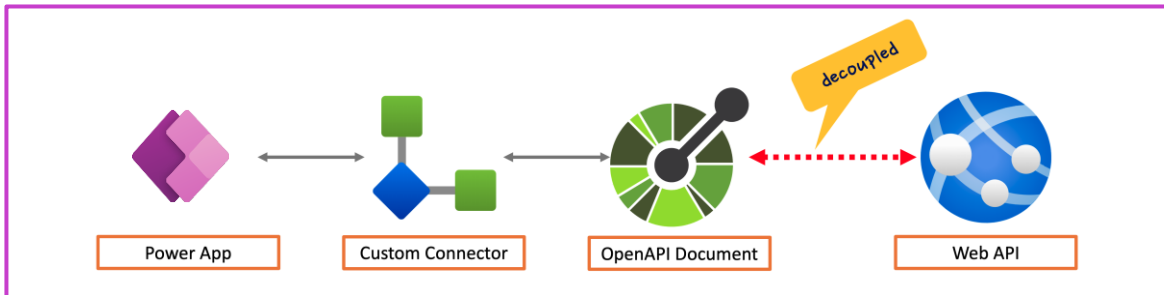
OpenAPI

OpenAPI defines a standard and programming language-agnostic interface description for web APIs.

It allows both humans and computers to discover and understand the capabilities of a service without having to access to source code, additional documentation, or inspect network traffic.

OpenAPI 2.0 (Swagger) JSON file.

Power Platform lets you instantly create a custom connector with this OpenAPI document.



© Copyright Microsoft Corporation. All rights reserved.

OpenAPI defines a standard and programming language-agnostic interface description for web APIs.

It allows both humans and computers to discover and understand the capabilities of a service without having to access to source code, additional documentation, or inspect network traffic.

Using an OpenAPI document describing a web API, the consumer can understand and communicate with the web API with minimal effort.

With the OpenAPI document, you don't have to know where the API is located or whether it's running, because the document contains all the information you need.

Power Platform lets you instantly create a custom connector with this OpenAPI document.

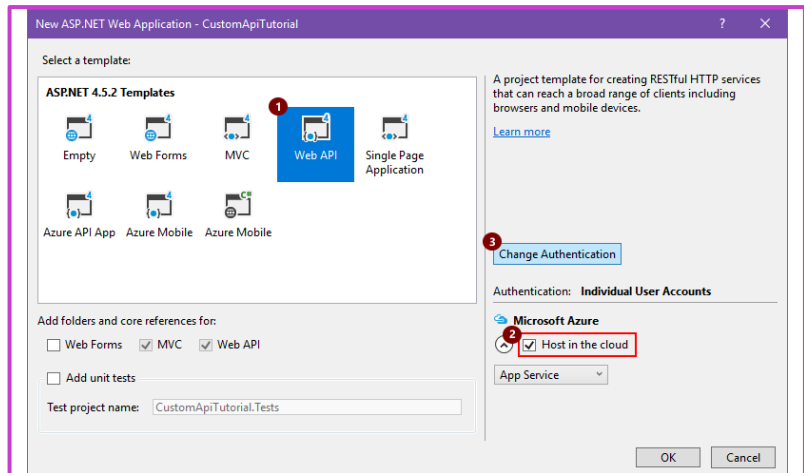
There are many advantages of using the OpenAPI document for your custom connector creation. But the following two benefits are the most crucial for citizen developers.

- **Remove dependencies:** With an OpenAPI document you can create a custom connector by yourself, without having to wait for somebody to create one for you from API Management. Plus, it is becoming more common for professional development teams to include OpenAPI document generation capabilities within the web API itself. This means you can download the OpenAPI document by visiting a URL and then create the custom connector with it.
- **Increase agility:** The cadence of your Power App development increases when your web API development team can hand you an OpenAPI document, and you do not need to rely on them for anything further to create and use a custom connector.

<https://learn.microsoft.com/connectors/custom-connectors/define-openapi-definition>

Create a custom connector from ASP.NET Core web API

- ASP.NET web app in the Azure App Service
- Use Swashbuckle to create an OpenAPI file
- OpenAPI `http://<web-api-app-root-URL>/swagger/docs/v1`
- Import the OpenAPI to create custom connector



© Copyright Microsoft Corporation. All rights reserved.

You can also create a custom connector by creating an ASP.NET web app in Azure

You can use Swashbuckle, an open-source tool, to create the OpenAPI file and then import the OpenAPI definition to create a custom connector

<https://learn.microsoft.com/connectors/custom-connectors/create-web-api-connector>

<https://learn.microsoft.com/aspnet/core/tutorials/web-api-help-pages-using-swagger>

OpenAPI vs Azure API Management

Connector	Azure API Management	OpenAPI
API Control	Centralized	Distributed
API Structure Layer	Complex	Simple
Extra Security Layer	Yes	No
Usage Control	Yes	No
Extra Cost	Yes	No
Architectural Complexity	Higher	Lower

© Copyright Microsoft Corporation. All rights reserved.

To create a custom connector, you can use either Azure API Management or an OpenAPI document. Which one should you choose?

The table lists some comparisons between Azure API Management and OpenAPI for custom connectors.

Module 3: OpenAPI definitions

© Copyright Microsoft Corporation. All rights reserved.

In this module, you'll learn about:
Adding Microsoft extensions to the custom connector

Learning Objectives

After completing this module, you will be able to:

- 1 Learn about OpenAPI extensions.
- 2 Use OpenAPI extensions.

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Create Power Automate OpenAPI custom connectors

<https://learn.microsoft.com/training/modules/custom-connectors-open-api>



© Copyright Microsoft Corporation. All rights reserved.

Create Power Automate OpenAPI custom connectors

<https://learn.microsoft.com/training/modules/custom-connectors-open-api>

OpenAPI extensions

The OpenAPI specification allows you to define extensions that augment the core definition to handle specific scenarios.

Microsoft has defined several extensions to the OpenAPI definition to allow you to configure specific features that are supported by custom connectors.

© Copyright Microsoft Corporation. All rights reserved.

The OpenAPI specification allows you to define extensions that augment the core definition to handle specific scenarios.

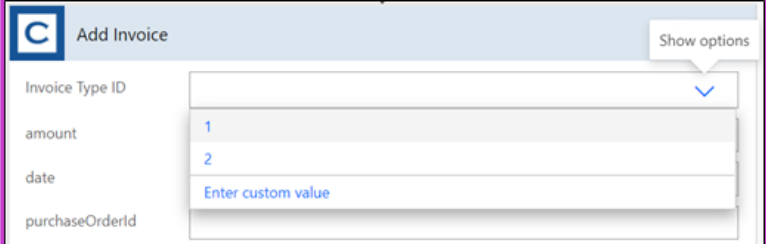
Microsoft has defined several extensions to the OpenAPI definition to allow you to configure specific features that are supported by custom connectors.

OpenAPI extensions

The primary use case for configuring these extensions is to make the connector easier to use.

For example, without dynamic values, makers would have to know internal unfriendly schema values or strings for type/lookup parameters.

By configuring dynamic values, you're helping to provide a user-friendly drop-down list for makers to select from.



The screenshot shows a web form titled 'Add Invoice' with a blue header bar containing a 'C' logo and a 'Show options' button. Below the header, there are four input fields: 'Invoice Type ID', 'amount', 'date', and 'purchaseOrderId'. The 'Invoice Type ID' field is open, displaying a dropdown menu with three options: '1', '2', and 'Enter custom value'. The 'amount' field contains the value '1', the 'date' field contains the value '2', and the 'purchaseOrderId' field is empty.

© Copyright Microsoft Corporation. All rights reserved.

The OpenAPI specification allows you to define extensions that augment the core definition to handle specific scenarios.

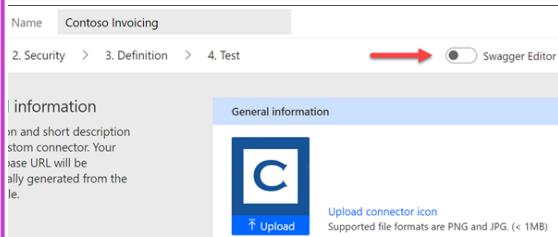
Microsoft has defined several extensions to the OpenAPI definition to allow you to configure specific features that are supported by custom connectors.

The primary use case for configuring these extensions is to make the connector easier to use. For example, without dynamic values, makers would have to know internal unfriendly schema values or strings for type/lookup parameters. By configuring dynamic values, you're helping to provide a user-friendly drop-down list for makers to select from.

Keep in mind that some extensions also require support from the API. For example, dynamic schema requires the API to support an operation to retrieve the schema. In cases where you don't control the API, it might not be possible to configure these types of extensions.

OpenAPI extensions

Identify extensions by looking for
`x-ms-<name>` in the OpenAPI definition
Add extensions using the Swagger editor



Example extensions

- `x-ms-capabilities`
- `x-ms-url-encoding`
- `x-ms-dynamic-values`
- `x-ms-dynamic-list`
- `x-ms-dynamic-schema`
- `x-ms-visibility`

© Copyright Microsoft Corporation. All rights reserved.

You can identify each extension by looking for **`x-ms-<name>`** in the OpenAPI definitions. You've likely already used some of these definitions when you configured a custom connector with the designer, such as when entering a summary on an action's parameter that uses the **`x-ms-summary`** extension to store the value in the definition.

Several extensions can only be edited either directly through the designer's Swagger editor or by exporting the connector's definitions and then editing and importing them back

- `x-ms-capabilities` - Used on the connector and operation level definition to indicate features that are offered.
- `x-ms-url-encoding` - Specifies encoding for a path parameter.
- `x-ms-dynamic-values` and `x-ms-dynamic-list` - Provide a list of options for the user to select input parameters for an operation.
- `x-ms-dynamic-schema` - Allows you to make parameters and response dynamic based on results of schema that is retrieved from an operation on the API.

Use OpenAPI extensions

Chunking is used to process large messages in an API

If the API supports chunking, then need to enable chunkTransfer in the OpenAPI definition

```
"x-ms-capabilities": {  
  "chunkTransfer": true  
}
```

© Copyright Microsoft Corporation. All rights reserved.

The x-ms-capabilities extension helps you configure which capabilities are offered by the connector at the connector level and operation level. Currently, Microsoft Power Platform custom connectors can be configured for the following options:

- chunkTransfer - Operation level

Enable chunk transfer

When handling messages, the connector runtime limits message content to a maximum size. This limit helps reduce overhead that is created by the storing and processing of large messages. To help handle messages that are larger than this limit, connectors can chunk a large message into smaller message chunks. That way, you can still transfer large content. When communicating with other services through connectors, the runtime can consume large messages but only in chunks. This condition means that connectors must also support chunking, and the underlying HTTP message exchange between the connector and these services must use chunking.

Module 4: Policy templates

© Copyright Microsoft Corporation. All rights reserved.

In this module, you'll:

- Learn about policy templates

Learning Objectives

After completing this module, you will be able to:

- 1 Discover how policies can modify the behavior of custom connectors at runtime.
- 2 Apply policy templates to a custom connector.

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Configure policy templates for custom connectors in Power Automate

<https://learn.microsoft.com/training/modules/policy-templates-custom-connectors>



© Copyright Microsoft Corporation. All rights reserved.

Configure policy templates for custom connectors in Power Automate

<https://learn.microsoft.com/training/modules/policy-templates-custom-connectors>

Policy templates

Policies allow you to modify the behavior of a custom connector at runtime.

You can use policies to perform data conversion, route requests, set parameter values, and more.

You can configure policies directly in the custom connector API properties file before import, or you can do it from the maker portal in the custom connector designer by applying policy templates.

© Copyright Microsoft Corporation. All rights reserved.

Policies allow you to modify the behavior of a custom connector at runtime.

You can use policies to perform data conversion, route requests, set parameter values, and more.

You can configure policies directly in the custom connector API properties file before import, or you can do it from the maker portal in the custom connector designer by applying policy templates.

<https://learn.microsoft.com/connectors/custom-connectors/policy-templates>

Use cases for policy templates

Route request

- Routes requests to a specified endpoint on the same service

Set Host URL

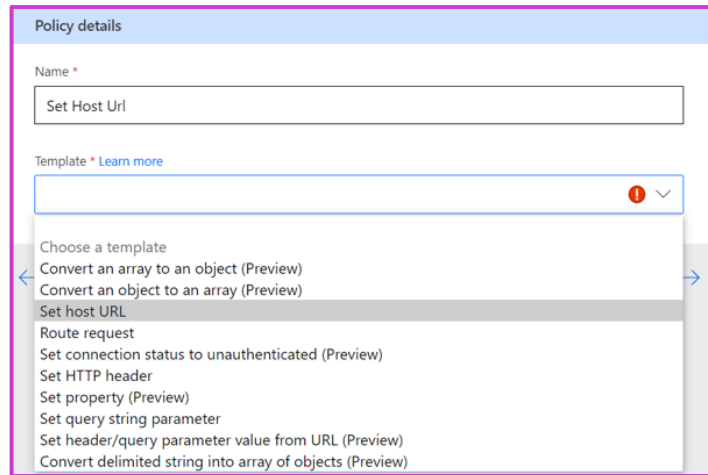
- Route calls to different endpoints
- Use expressions to access runtime values

Set HTTP header

- Override or add information to a request or response
- Enable CORS for API Management connectors

Set Query String Parameter

- Adds or updates value of request query string parameter



© Copyright Microsoft Corporation. All rights reserved.

Common policy use cases are:

- Set host URL - By default, the host URL is hardcoded into the connector configuration. A policy, combined with a connection parameter, allows the host URL to be specified every time you create a connection by using the connector. For example, this use case could handle a dev, test, and production version of the system API.
- Set header - Use to promote data from the connection parameters, query, or body of the request to the header. Commonly, this process is done to accommodate APIs that want specific information configured in the header. For example, this use case could be a correlation ID to track the full business process that is being performed.
- Set query parameter - Use for handling default values, if necessary, but the maker doesn't configure one. For example, if an API required a top parameter for how many records to return by using a policy, you could set a default value to be used if the maker doesn't configure one.

<https://learn.microsoft.com/connectors/custom-connectors/policy-templates/routerequesttoendpoint/routerequesttoendpoint>

<https://learn.microsoft.com/connectors/custom-connectors/policy-templates/dynamichosturl/dynamichosturl>

<https://learn.microsoft.com/connectors/custom-connectors/policy-templates/setheader/setheader>

<https://learn.microsoft.com/connectors/custom-connectors/policy-templates/setqueryparameter/setqueryparameter>

<https://learn.microsoft.com/connectors/custom-connectors/policy-templates/expressions/expressions>

<https://learn.microsoft.com/azure/api-management/enable-cors-power-platform>

Module 5: Write code in a custom connector

© Copyright Microsoft Corporation. All rights reserved.

In this module, you'll:

- Learn about policy templates

Learning Objectives

After completing this module, you will be able to:

- 1 Discover how custom code can transform request and responses
- 2 Create custom code in a custom connector.

© Copyright Microsoft Corporation. All rights reserved.

Microsoft Learn module

Use custom code in custom connectors for Microsoft Power Platform

<https://learn.microsoft.com/en-us/training/modules/custom-code-connectors/>



© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/en-us/training/modules/custom-code-connectors/>

Custom code

Use custom code to implement advanced transformation of the input and output of a connector's actions.

Developers can implement custom code to go beyond what policy templates are capable of.

Custom code logic takes complete control over the request and response transformation, including the task of invoking the operation on the underlying service.

© Copyright Microsoft Corporation. All rights reserved.

Implement custom code

To implement custom code, you'll need to create a class named `Script`, which must inherit from the abstract base class **`ScriptBase`**.

```
public class Script : ScriptBase
{
    public override Task<HttpResponseMessage>
    ExecuteAsync()
    {
        // Your code here
    }
}
```

© Copyright Microsoft Corporation. All rights reserved.

ScriptBase class has a Context public property

OperationId - This property is most useful when you need to identify which operation is being processed so that your logic can provide the correct transformations.

Request - This method is the `HttpRequestMessage` for the operation that's being processed. If you need to transform the request, you modify this method, and then pass it to `SendAsync` to send the request to the service.

Logger - This method provides you with an instance of an `ILogger` that you can use to log diagnostic information from your code to the Code Logs for the connector operation.

SendAsync - Use this property to send an HTTP request to the service. You should use this method instead of making your own `HttpClient.SendAsync` call.

© Copyright Microsoft Corporation. All rights reserved.

The **ScriptBase** class also has a **Context** public property, which is available to your logic to provide context for the request that's being processed. The **Context** property is of the type **IScriptContext**

Add code to the connector

- Select the Code tab and then turn on the Code Enabled toggle.

The screenshot shows the 'Code' tab of the 'Contoso Invoicing - Extensions' connector configuration. The breadcrumb navigation at the top indicates the current step is '4. Code'. The left sidebar contains instructions for the 'Code' step. The main area has a text input field for pasting code and an 'Upload' button. Below this, the 'Code Details' section is visible, and the 'Code Enabled' toggle switch is turned on and highlighted with a red rectangular box.

Connector Name: Contoso Invoicing - Extensions

1. General > 2. Security > 3. Definition > **4. Code** > 5. Test

Swagger Editor Update connector Close

Code

This step is optional. Custom code transforms request and response payloads beyond the scope of policy templates. You can either paste in your code or upload a file with your code. Your code must be in C#, have a maximum execution time of 5 seconds, and can't be more than

Paste your code in the window below manually or by clicking the upload button below.

Upload

Code Details

Code Enabled

Operations

List of actions and triggers to which the policy will apply to. If no operation is selected, this policy will apply to all operations

© Copyright Microsoft Corporation. All rights reserved.

Select the operations to apply the code

Paste your code in the window below manually or by clicking the upload button below.

Upload

Code Details Code Enabled

Operations
List of actions and triggers to which the policy will apply to. If no operation is selected, this policy will apply to all operations.

GetLastInvoice

Actions

- ☐ AddInvoice
- ☐ GetInvoice
- ☐ GetInvoiceSchema
- ☒ GetLastInvoice
- ☐ ListInvoices

© Copyright Microsoft Corporation. All rights reserved.

Paste in the custom code

Connector NameContoso Invoicing - Extensions

1. General > 2. Security > 3. Definition > 4. Code > 5. TestSwagger EditorUpdate connectorClose

Code

This step is optional. Custom code transforms request and response payloads beyond the scope of policy templates. You can either paste in your code or upload a file with your code. Your code must be in C#, have a maximum execution time of 5 seconds, and can't be more than 1MB. [Learn more](#)

Paste your code in the window below manually or by clicking the upload button below.

Upload

Code Details

Code Enabled

Operations

List of actions and triggers to which the policy will apply to. If no operation is selected, this policy will apply to all operations.

GetLastInvoice

```
1 public class Script : ScriptBase
2 {
3     public override async Task<HttpResponseMessage> ExecuteAsync()
4     {
5         // Check if the operation ID matches what is specified in the OpenAPI definition of the co
6         if (String.Equals(this.Context.OperationId, "GetLastInvoice", StringComparison.Ordinal)
7         {
8             this.Context.Request.RequestUri = ReplaceUri(this.Context.Request.RequestUri, "GetLast
9             return await this.HandleGetLastInvoiceOperation().ConfigureAwait(false);
10         }
11         else
12         {
13             //pass-thru any other operation to the API directly
14             return await this.HandleForwardOperation().ConfigureAwait(false);
15         }
16     }
17     private Uri ReplaceUri(Uri original, string fromValue, string toValue)
```

© Copyright Microsoft Corporation. All rights reserved.

Learning Path 8 practice labs



Azure Functions, Webhooks, and Custom Connectors

Lab 9: Integration with Azure

© Copyright Microsoft Corporation. All rights reserved.

In this lab you will use the event publishing capability of Microsoft Dataverse. When a permit results in changing the size of the build site, an external taxing authority needs to be notified so they can evaluate if additional taxing is required. You will configure Microsoft Dataverse to publish permits with size changes using the Webhook. To simulate the taxing authority receiving the information you will create a simple Azure function to receive the post.

In this lab you will also build a custom connector that can be used from Power Apps and Power Automate. Custom connectors describe existing APIs and allow them to be used easily. In this lab, you will build an API that has common calculations used by inspectors so that they can be used by applications. After building the API, you will create a custom connector definition to make it available to Power Apps and Power Automate.

Summary



- Custom connectors describe REST APIs
- Custom connectors can be used like any other connector in canvas apps and power Automate cloud flows
- Azure Functions with HTTP triggers can be consumed using custom connectors
- Custom connectors can easily be created for APIs managed by Azure API Management
- OpenAPI 2.0 definitions can be extended for custom connectors
- The behavior of a custom connector can be modified at runtime by a policy template
- Use custom code for more advanced request and response transformations

© Copyright Microsoft Corporation. All rights reserved.

Modules

1. Get started with custom connectors
2. Azure API Management and custom connectors
3. Open API definitions
4. Policy templates



© Copyright Microsoft Corporation. All rights reserved.